Power BI Translytical Task Flow and Fabric User Defined Functions

Nico Jacobs, U2U
@SQLWaldorf

When BI becomes actionable

- Business intelligence analyzes and reports on data
- The insights brought by BI can allow people to act on this
  - Change product portfolio
  - Run customer retention programs, …
- But sometimes people want to act on the data shown in the report
  - Change budget allocations
  - Adjust work schedules
  - Finetune discount proposals, …

Power BI data modifications

- Power BI natively doesn't support data modifications
  - No INSERT, UPDATE nor DELETE in DAX nor Power Query
- But there were workarounds
  - Power Query could use e.g. Python transformations to materialize datasets to disk
  - Power Automate visuals trigger a flow that can modify data
- They did not allow for interaction with regular Power BI visuals

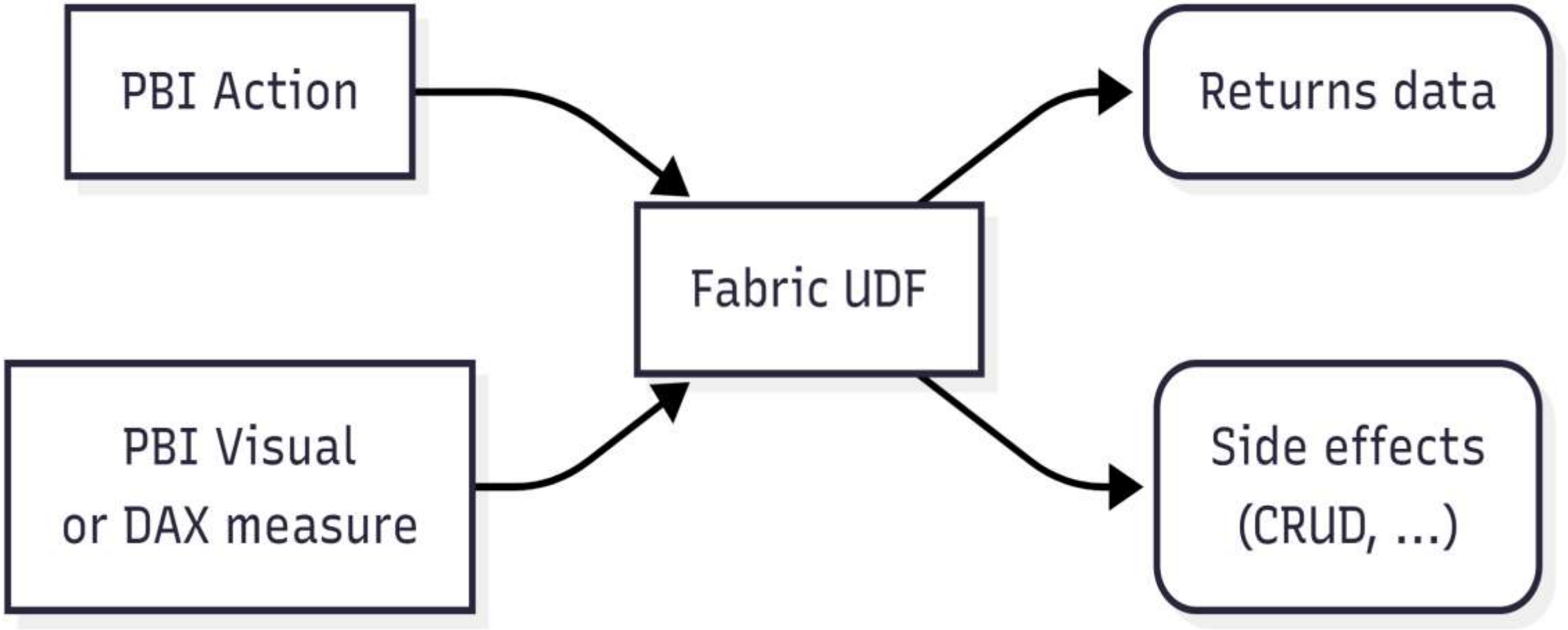
Analytical + Transactional

Translytical

Translytical Task Flow

- Translytical task flow combines the analytical power of Power BI with transactional procedures that 'act'
  - INSERT, UPDATE or DELETE data
  - Start a Notebook, Pipeline, …
  - Send an email
  - …
- Remaining questions:
  1. Where do we trigger the method?
  2. Where do we write the method? In which language?
  3. Where does it run? Who pays for it?
  4. With whose permission does it run?
  5. How can we pass in parameters?
  6. Can we (besides the side-effects) return results from the method?

Translytical task flow



Demo

Building a "Hello World" Translytical Task Flow:

- Action
- UDF in Fabric
- Text slicer
- Using data model as input

Recap

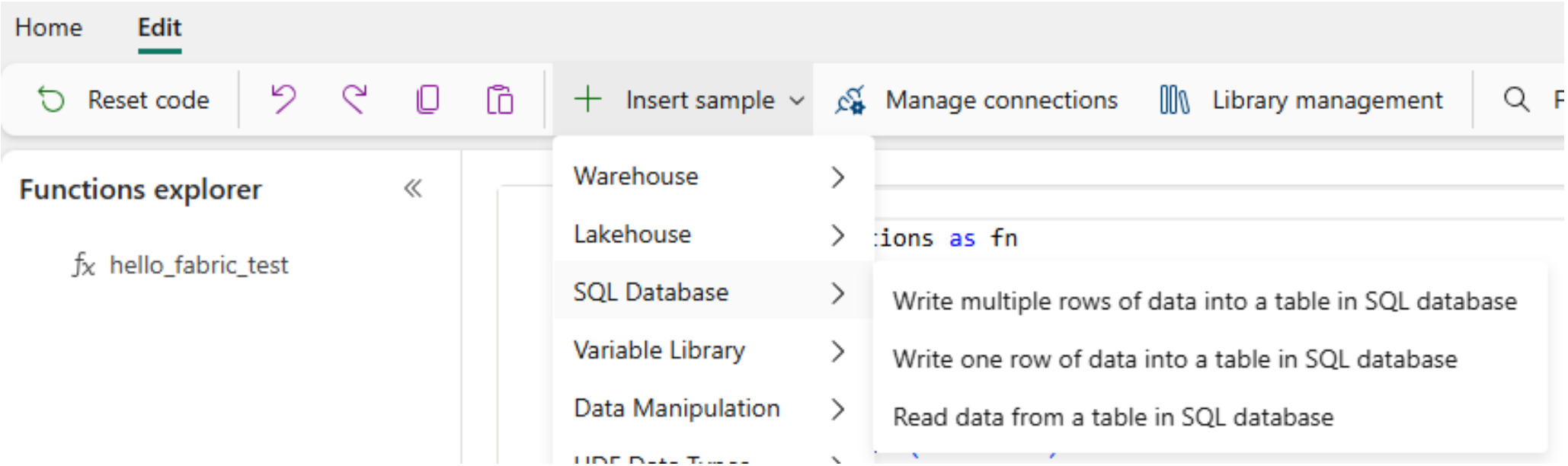
- Buttons in Power BI have a **Data Function** action
- Select the User Data Function (UDF) that you made in Microsoft Fabric
- Feed in the data for the function parameters
  - Either from a Power BI Visual (List, Text or Button Slicer)
  - Or from an explicit or implicit measure
- When selecting a slicer as input, please name your slicer for a readable identifier
- Power BI will refresh the report when activating the UDF
  - Deselect this option if the action does not change the report itself

Next step: Modify data

- How can the Python function access data?
- For a variety of data sources in Fabric, the UDF can create and manage a data source connection

Using connections

- To generate a user data function, you can use templates provided in the portal



Permissions

- The Python code runs with the permissions of the Entra user running the Power BI report

```python
def read_username(sqlDB: fn.FabricSqlConnection)-> str:
    query = "SELECT USER_NAME();"
    connection = sqlDB.connect()
    cursor = connection.cursor()
    cursor.execute(query)
    # Fetch all results
    results = []
    for row in cursor.fetchall():
        results.append(row)

    # Close the connection
    cursor.close()
    connection.close()
```

Test

Succeeded

Output (str)

This ran as SQL Database user ('nicoj@u2upbidev.onmicrosoft.com',).

Logs

Demo

Data connections and sample code

Using extra Python packages

- Python's power is based on the many packages that extend the language
- However, not all these packages are pre-installed

```python
from faker import Faker

udf = fn.UserDataFunctions()

@udf.function()
def hello_random() -> str:
    fake = Faker()
    name = fake.name()
    return f"Welcome, {name}"
```

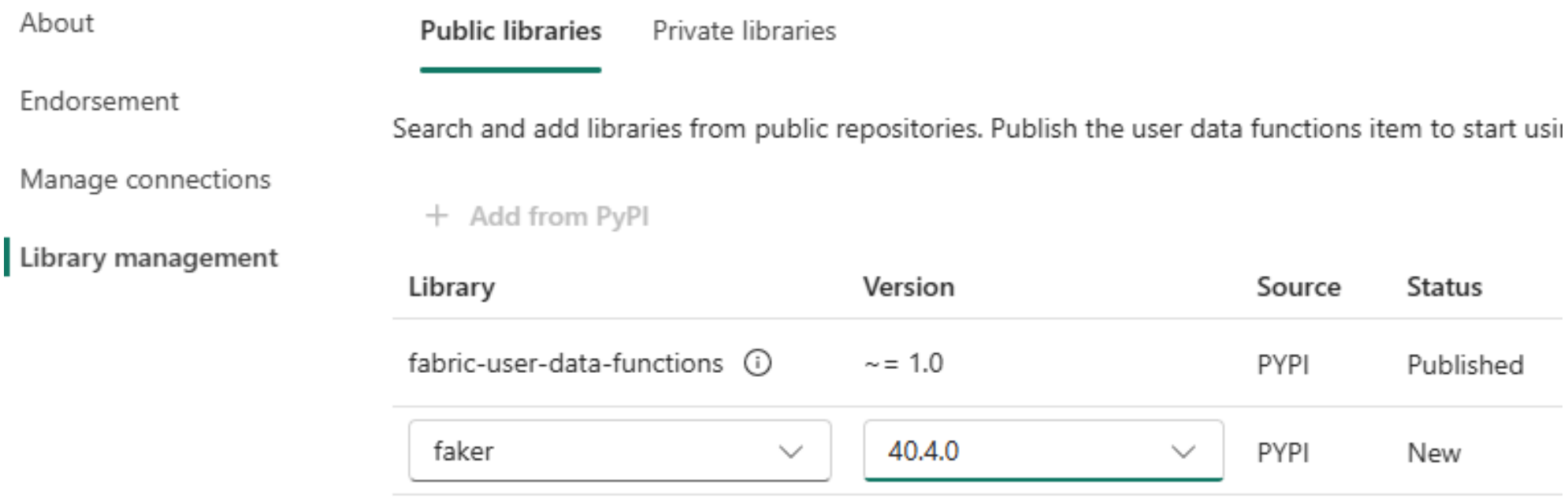
null

Logs

```
No module named 'faker'
----------------
ModuleNotFoundError
```

Installing extra packages

- In a self-hosted environment conda or pip can be used to add extra packages to the environment
- In Fabric UDFs you can add Pypi packages via the library management interface
- Custom Wheel files can be loaded as well



Demo

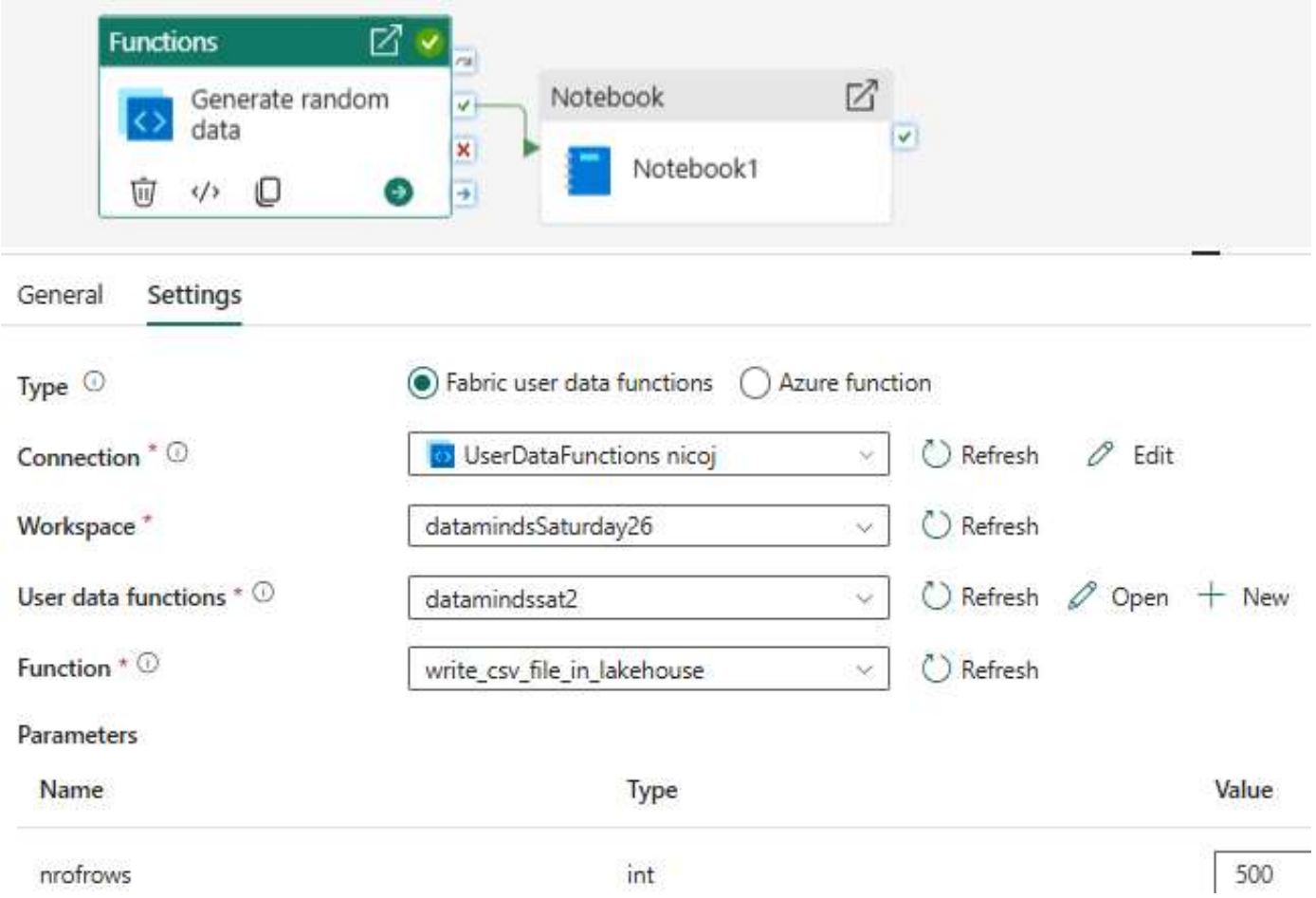
Loading the faker library

Limitations

- UDFs must have an explicit type for parameters and return value
- Limited set of data types (both input and output)
- No underscores in parameter names
- No default values for parameters
- Functions are by default blocking (synchronous)
  - Use async keyword when doing time consuming operations

Fabric Pipelines

- Fabric pipeline provides orchestration for various activities
- With the Functions activity either Azure Functions or Fabric UDFs can be called



Fabric Notebooks

- Notebooks in Fabric allow you to run Python or Spark code
- With the notebookutils library you can access various parts of Fabric
  - Amongst others also the Fabric UDFs

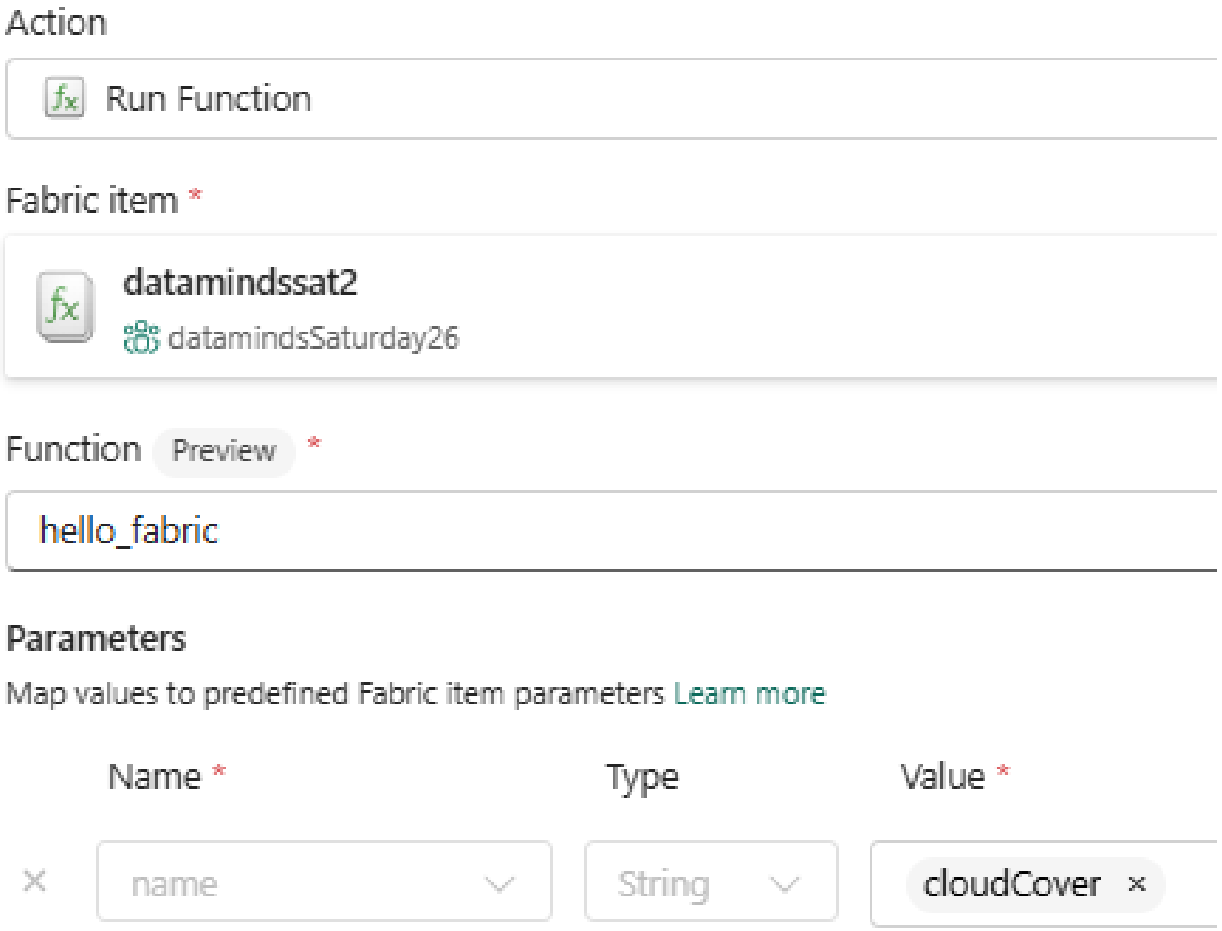
```
1   myFunctions = notebookutils.udf.getFunctions('datamindssat2')
2   myFunctions.write_csv_file_in_lakehouse(nrofrows= 300, filename = 'nbdemo.csv'
```

✓ 7 sec - Command executed in 7 sec 151 ms by Nico Jacobs on 2/20/2026, 2:08:05 PM

```
'File nbdemo.csv with 300 rows of random data was written to the Lakehouse.'
```

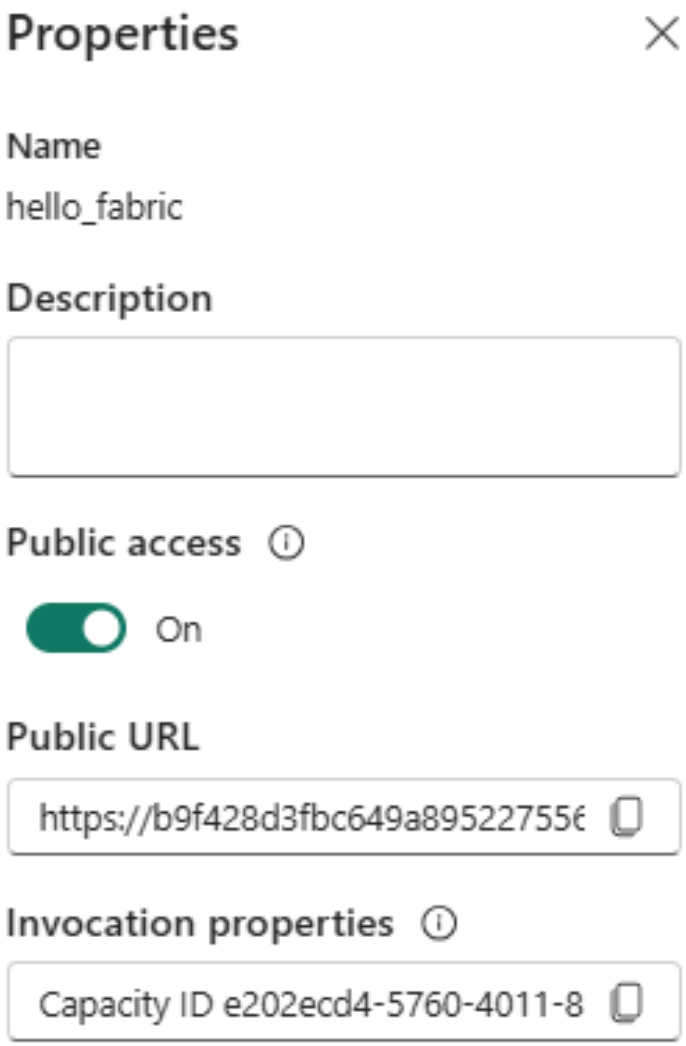
Fabric Activator

- Fabric Activator is an orchestrator for streaming data
  - It monitors the properties of one or more objects
    - E.g., the Location object can have a temperature and humidity property
  - When Boolean conditions on these properties are met, they can trigger an action
    - E.g., When the average temperature in the last 5 minutes rises above 40° C
  - Possible actions are: send an email, teams message, run a Fabric function, …



REST endpoint

- You can also call the function via REST services
  1. Create and authorize Entra app registration
  2. Make the UDF public (default) and copy URL
  3. In your app, authenticate with the app registration
  4. Get bearer token
  5. Call the web service



Demo

Conclusions

Python	• Microsoft Fabric User Data Function (UDF) is a Python function, stored and executed in Microsoft Fabric
Translytical side-effects	• These functions take input parameters, return a value and can cause side-effects such as inserts, updates and deletes
Reusable	• The functions can be called from Power BI, Fabric Pipelines, Notebooks and Activators, as well as from external REST calls
Environment	• You can manage the connections and packages used by the UDFs
Input	• When calling them from Power BI, input values can come from small number of visuals, or from measures

Thanks!

- Github code snippets
- Linked-in connection